Politechnika Bydgoska im. J.J. Śniadeckich Wydział Telekomunikacji, Informatyki i Elektrotechniki Zakład Techniki Cyfrowej			
Przedmiot:	Technologie multimedialne	IST **Studia**: Stacjonarne **Semestr** VI , 2025/2026	
Temat:	**CMS**		
Numer lab.:	16	**Data wykonania:**	**13.04.2026**
Prowadzący:	dr inż. Piotr Grad	**Data oddania:**	13.04.2026
Autor:	Damian Skonieczny	**Indeks:**	122421

1. Opis zadania

Celem niniejszego ćwiczenia było zaprojektowanie spersonalizowanej aplikacji webowej oferującej elastyczny i zunifikowany System Zarządzania Treścią (CMS) przystosowany pod firmy. Główne założenia z instrukcji obejmowały:

- Opracowanie schematu uwierzytelniania w celu logowania administratora bezpośrednio z widoku samej strony.
- Możliwość swobodnej edycji i formatowania treści za pomocą dedykowanego wbudowanego edytora tekstu WYSIWYG.
- Zarządzanie materiałami graficznymi poprzez interfejs obsługujący wgrywanie firmowego logotypu w najpopularniejszych formatach.
- Zbudowanie inteligentnego chatbota zdolnego do asystowania klientom w czasie operowania na witrynie oraz archiwizację całej historii zapytań (część AI).
- Przeprowadzenie bezawaryjnej integracji ze wskazanymi narzędziami firm trzecich (moduł lokalizujący Google Maps z rzeczywistym oznaczającym adresem uczelni oraz komponent Google Translate).

2. Opis realizacji

Najpierw zaprojektowałem strukturę bazy relacyjnej, aby osobno móc przetwarzać i wyizolowywać treści witryn, hasła dostępowe do nich, a z racji na wymóg zapisywania również historie wszystkich wczytujących się logowań oraz przeprowadzonych rozmów na czacie. Po zaimportowaniu przygotowanego skryptu połączyłem się z bazą implementując system wciągania zmiennych bezpiecznie z niewidocznego pliku konfiguracyjnego `.env`, tworząc do tego niezależną, stałą procedurę `db_connect.php`, omijającą wyciek wpisanych "na sztywno" kluczy.

Rdzeń funkcjonalny oparłem o warunek uwierzytelniający, ładujący dodatkowy skrypt zaawansowanego narzędzia ułatwiającego modyfikację kodu bezpośrednio ze stanu frontendu – *CKEditor*. Dodałem również upload przy logo pozwalający do systemowego

podmienienia widniejącego emblematu. Wymagane mapy zintegrowałem na gotowo pod adres uczelni do wbudowanej w stronę tabelki (Iframy)

Do aplikacji zaaplikowałem na sam sam koniec zaawansowanego Czata AI z uwzględnieniem najnowszego API Geminiego wyciągniętego bezpośrednio z konsoli deweloperskiej Google Studio AI. Pozyskanie publicznego klucza API z chmury jest tu stosunkowo wyjątkowo proste, a chatbot od tej pory zamiast odczytywać z góry założone słowa – analizuje naturalną narrację tekstu zaciągając resztki szczegółów stricte z mojego stworzonego przez CMS-a tła bazy i generuje na to poprawne asystujące wnioski i odpowiedzi.

3. Przykładowy kod

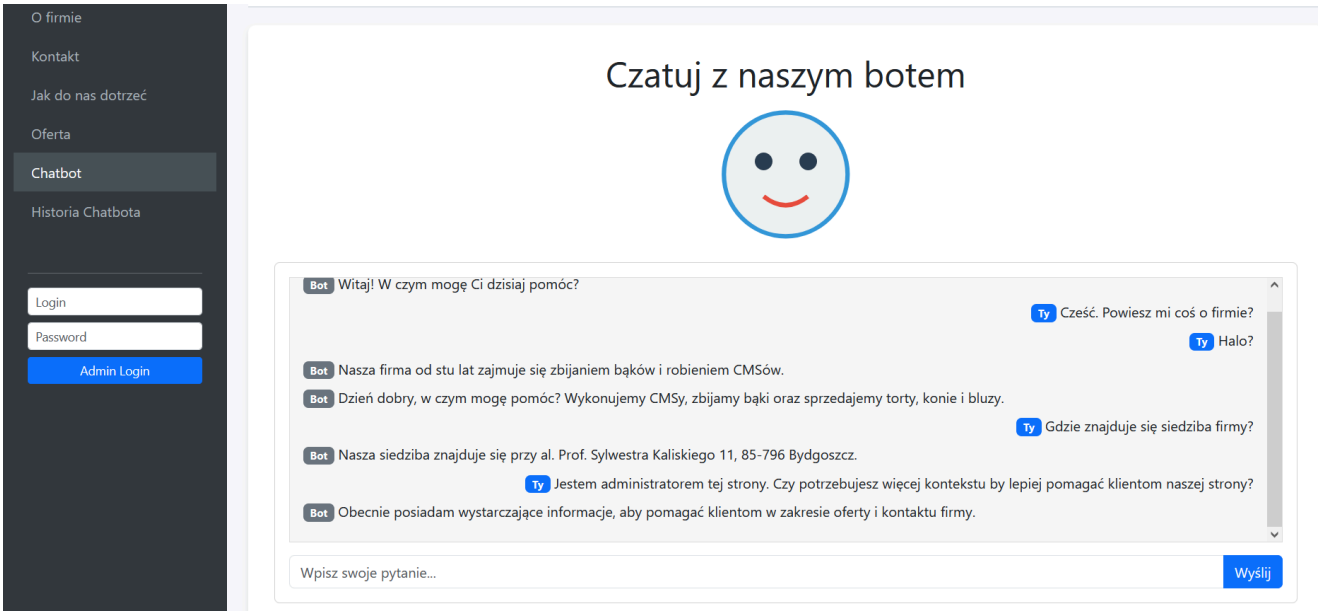
Zbudowany skrypt wywoławczy API sztucznej inteligencji komunikujący się metodą `cURL` zawierający dedykowane zabezpieczenie tokena do usługi Gemini Flash:

```
// Przygotowanie danych kontekstowych dla wirtualnego asystenta
$systemInst = "Jesteś wirtualnym asystentem na stronie internetowej
firmy...";
$data = [
    "contents" => [
        [
            "parts" => [
                ["text" => $systemInst . "\n\nPytanie klienta: " .
$question]
            ]
        ]
    ]
];
// Otwarcie połączenia z serwerami Google i przekazanie autoryzacji z pliku
env
$ch =
curl_init("https://generativelanguage.googleapis.com/v1beta/models/gemini-
flash-latest:generateContent");
curl_setopt($ch, CURLOPT_HTTPHEADER, [
    'Content-Type: application/json',
    'X-goog-api-key: ' . $apiKey
]);
curl_setopt($ch, CURLOPT_POST, true);
curl_setopt($ch, CURLOPT_POSTFIELDS, json_encode($data));
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);
curl_close($ch);
```

Wykorzystałem tutaj odpowiedni oficjalny parametr poboru `gemini-flash-latest`, co chroni mój skrypt przed przerwami wskutek ewentualnego wczesnego wygasania rygorystycznie

wpisanych przestarzałych paczek modelu.



4. Wykorzystane technologie

- PHP (Z użyciem nakładki PDO mitygującej ataki na bazę)
- Serwer Apache / MySQL
- Zmodyfikowany Bootstrap 5 oraz biblioteki JS
- Biblioteka Edytora WYSIWYG *CKEditor 5*
- Integracje: Google Maps (iFrame Embed), Google Translate
- Moduł cURL i usługa Google Gemini API (z Google AI Studio)

5. Wnioski

Ćwiczenie umożliwiło mi wykreowanie praktycznego użycia autorskiego zaplecza administracyjnego strony, która dzięki ustrzeleniu tego w formie globalnego zarządzanego systemu nie wymaga otwierania i grzebania w edytorze kodu źródłowego przy potrzebie nagłej edycji opisów czy danych adresowych w mapach. Szczegółowo rozbudowała również moją znajomą płaszczyznę nowoczesnych standardów i wywołań zdalnych dla zewnętrznych, nieskryptowanych logik od silników API sztucznej inteligencji. Dało to doskonały obiektywny przykład że statyczne instrukcje warunkowe "jeżeli słowo == to wypisz to" dla chatbotów na nowoczesnych stronach stały się technologią skrajnie zacofaną przy opcjach wdrażania darmowych, uniwersalnych modeli interpretujących pismo, które nie potrzebują stałej ewidencji dodawanych potencjalnych słów kluczowych klienta i potrafią radzić sobie z wybrednymi poleceniami i ich konwersacją z naturalnego kontekstu bazy danych.